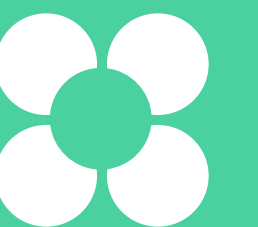


Переменные многопоточной программы

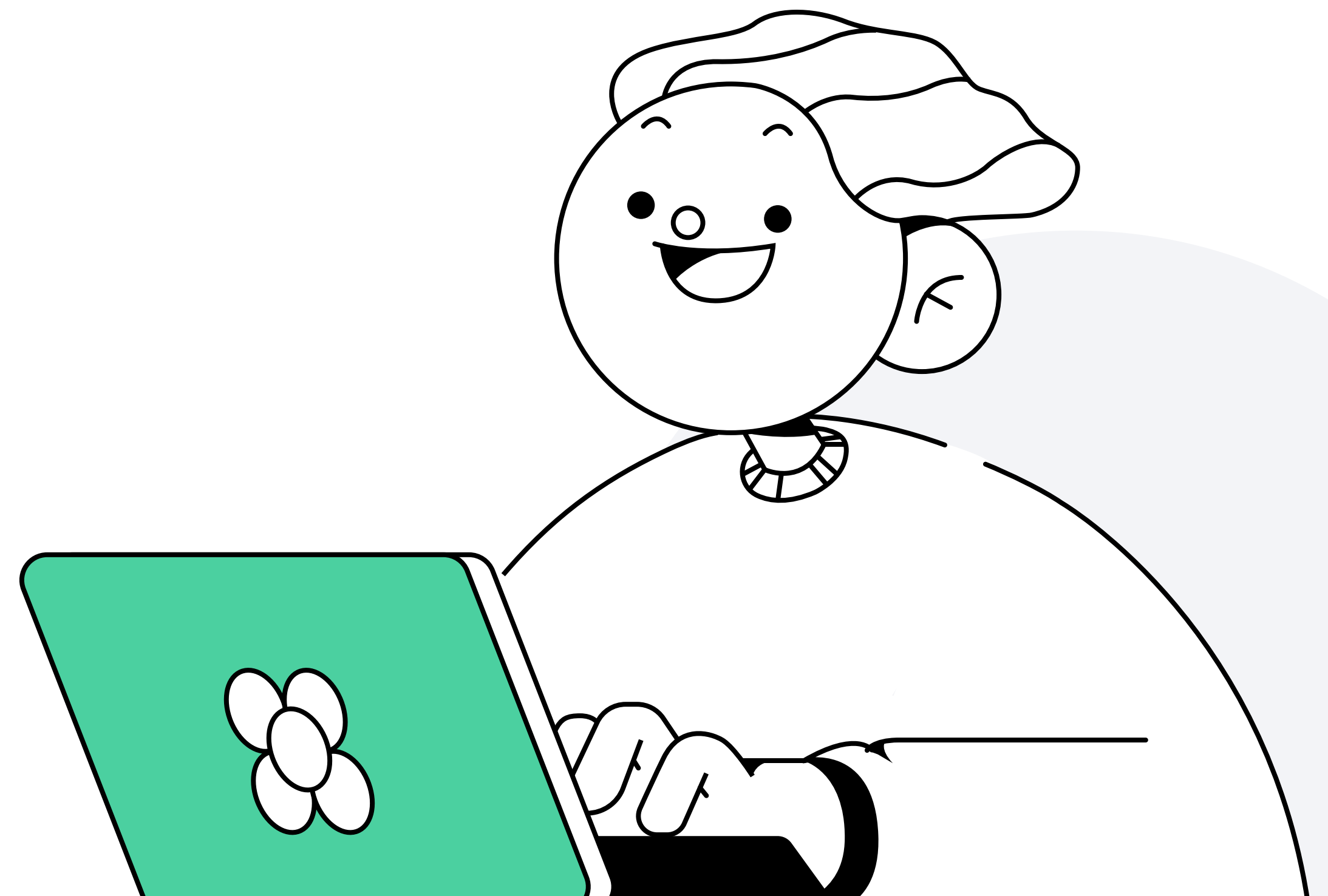
Филипп Воронов
Teamlead, поиск VK



План занятия

① Volatile

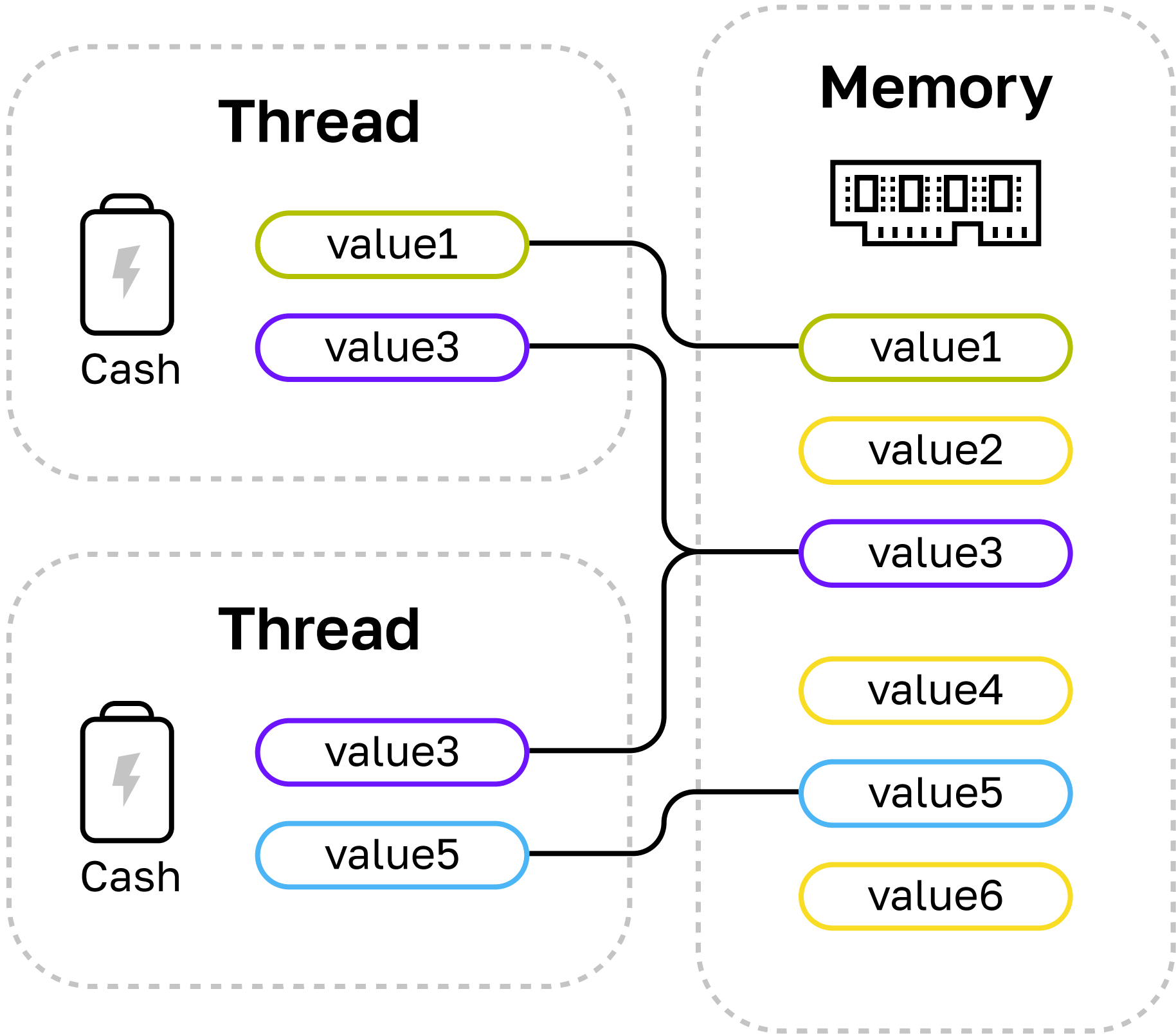
② Atomics



Volatile

Филипп Воронов
Teamlead, поиск VK

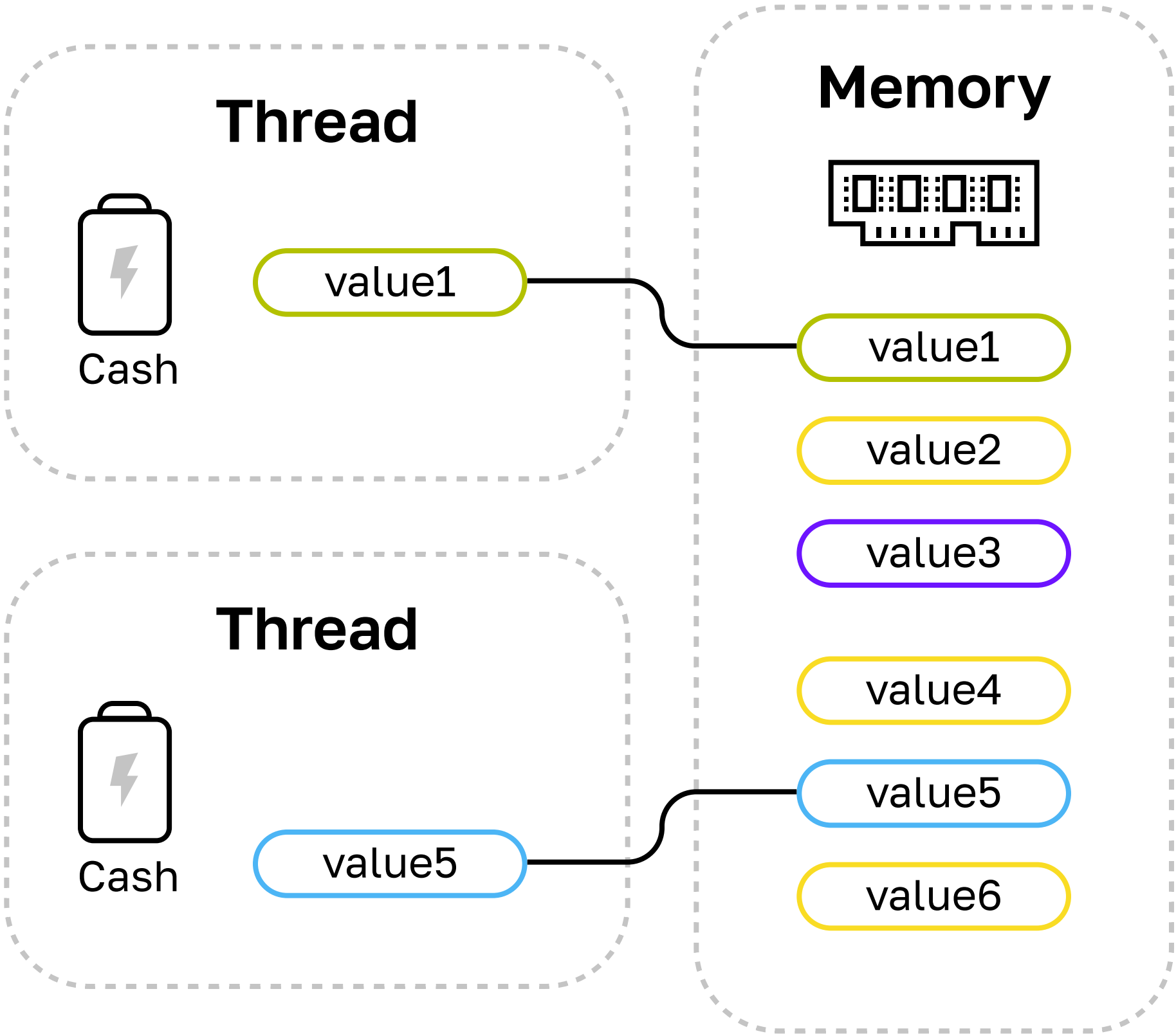
Volatile



```

int value3;

```



```

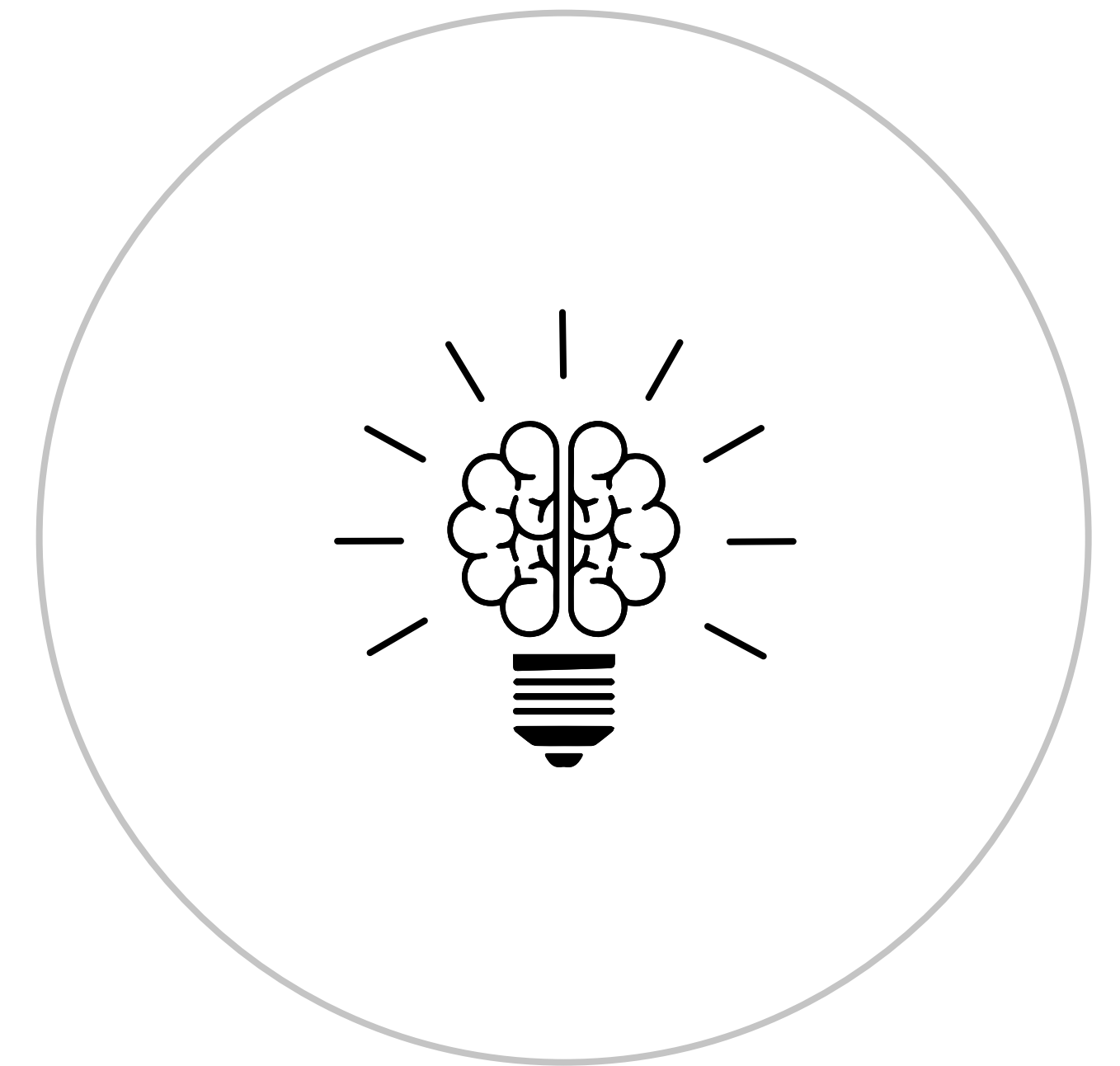
volatile int value3;

```

Volatile

Volatile-переменные
обеспечивают:

- Атомарность чтения и атомарность записи
- Запрет кеширования



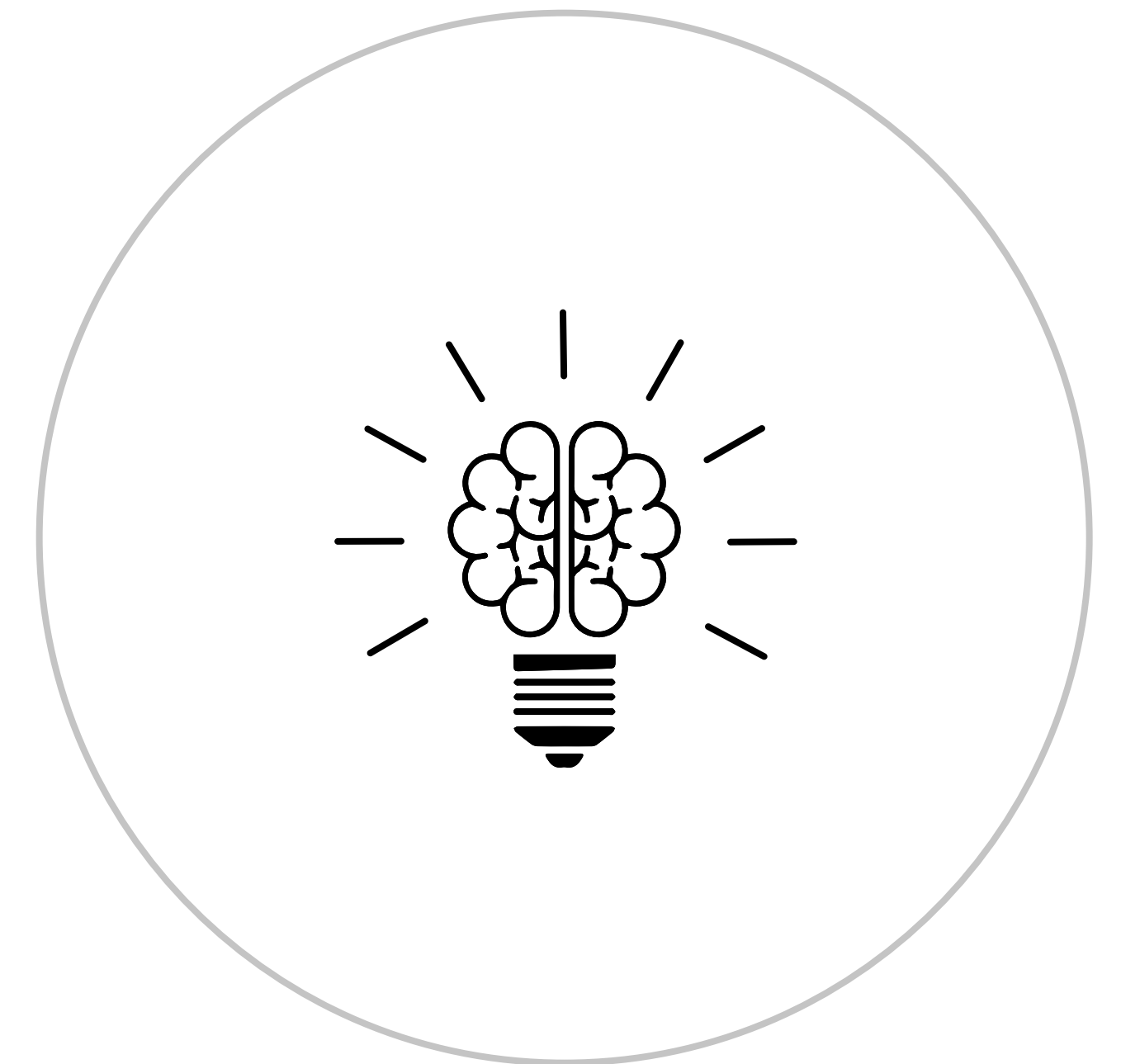
Atoms

Филипп Воронов
Teamlead, поиск VK

Atomics

Атомики могут выполнять две операции атомарно, то есть как неделимую операцию

- AtomicBoolean
- AtomicInteger
- AtomicLong
- AtomicReference



Atoms

Популярные методы атомиков:

get

→ получение текущего значения

set

→ установка нового значения

getAndSet

→ установка нового значения и получение старого значения

compareAndSet

→ установить новое значение, если текущее совпадает с ожидаемым

Atoms

Популярные методы `AtomicInteger` и `AtomicLong`:

`getAndIncrement`

→ получить текущее значение и добавить единицу

`getAndDecrement`

→ получить текущее значение и отнять единицу

`getAndAdd`

→ получить текущее значение и прибавить переданное число

`addAndGet`

`incrementAndGet`

`decrementAndGet`

Atoms

Напишем класс **Account**, который хранит деньги клиента банка:

```
class Account {  
    AtomicLong money = new AtomicLong(0);  
  
    public void transfer(long diff) {  
        System.out.printf("%s переводит вам %d рублей\n", Thread.currentThread().getName(), diff);  
        long result = money.addAndGet(diff);  
        System.out.printf("Теперь у вас на счету %d рублей\n", result);  
    }  
  
    public long getCurrent() {  
        return money.get();  
    }  
}
```

Atoms

Эмулируем ситуацию одновременных переводов
на 200, 300 и 500 рублей:

```
public static void main(String[] args) throws InterruptedException {  
    Account account = new Account();  
    Thread thread1 = new Thread(null, () -> account.transfer(500), "Клиент 1");  
    Thread thread2 = new Thread(null, () -> account.transfer(200), "Клиент 2");  
    Thread thread3 = new Thread(null, () -> account.transfer(300), "Клиент 3");  
  
    thread1.start();  
    thread2.start();  
    thread3.start();  
  
    thread3.join();  
    thread2.join();  
    thread1.join();  
    System.out.println("\nБаланс после всех переводов: " + account.getCurrent());  
}
```

Клиент 2 переводит вам 200 рублей

Теперь у вас на счету 200 рублей

Клиент 3 переводит вам 300 рублей

Теперь у вас на счету 500 рублей

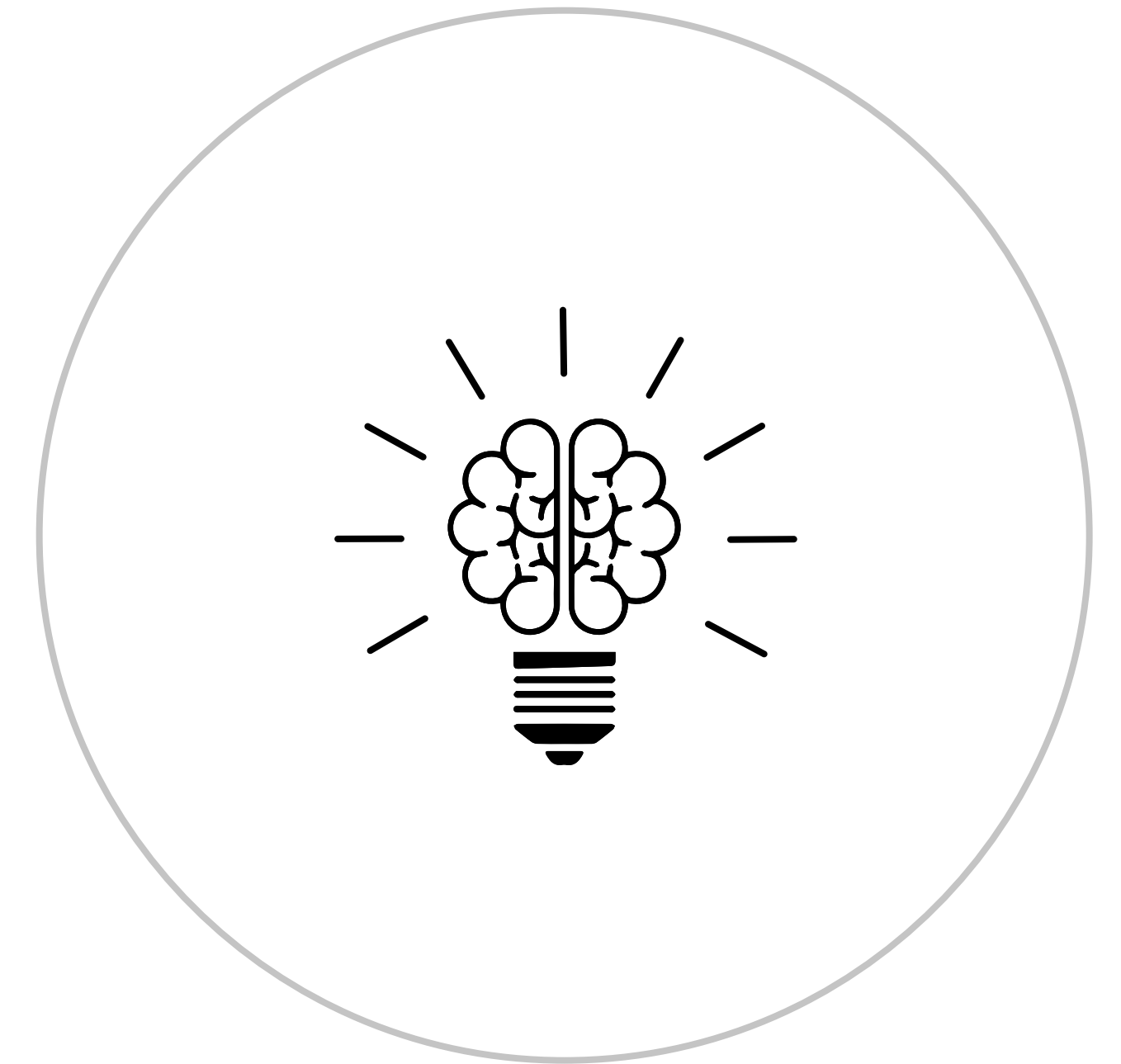
Клиент 1 переводит вам 500 рублей

Теперь у вас на счету 1 000 рублей

Баланс после всех переводов: 1 000

Atoms

- ❗ **AtomicAdder** позволяет вместо реального частого сложения просто хранить слагаемые. Сама же операция сложения выполняется только по запросу



Atomics

Небольшой пример использования **LongAdder**:

```
public static void main(String[] args) throws InterruptedException {  
    ExecutorService executorService =  
    Executors.newFixedThreadPool(Runtime.getRuntime().availableProcessors());  
    LongAdder stat = new LongAdder();  
    IntStream.range(0, 500)  
        .forEach(i -> executorService.submit(() -> stat.add(i)));  
  
    executorService.awaitTermination(3, TimeUnit.SECONDS);  
    System.out.println("\nРезультат: " + stat.sum());  
    executorService.shutdown();  
}
```

Добавляем расчёт

Считаем весь
результат